

diziAssist

Note d'architecture & justification des choix techniques

Développeur Full Stack Junior — IA, Automatisation & Produits Digitaux
Document de cadrage préalable au développement — 29 juillet 2026

1. Contexte et contraintes

Construire un MVP de suivi des actions issues des comptes rendus de réunion. Le produit doit permettre de saisir un compte rendu, d'en extraire des actions structurées, de les faire valider par un humain, puis de les suivre dans un tableau de bord.

Contrainte	Implication sur les choix techniques
6 h de travail effectif maximum	Écarter toute stack imposant du câblage non productif
Remise le 30/07 à 20 h GMT	Priorité absolue à la livraison d'un périmètre fini
Fonctionne sans IA obligatoirement	Extraction déterministe par règles en socle, IA en surcouche
Persistence après actualisation	Base de données réelle, pas de stockage navigateur
Aucune clé API exposée (pénalité majeure)	Appels LLM exclusivement côté serveur
Application lançable (pénalité majeure)	Installation sans compte externe ni service tiers
Code maîtrisé (pénalité majeure)	Aucune technologie découverte la veille

Répartition du barème et lecture stratégique

Poste	Points	Levier principal
Fonctionnalités obligatoires	25	Périmètre restreint mais entièrement terminé
Qualité & structure du code	15	TypeScript strict, validation centralisée, tests ciblés
Architecture simple et évolutive	10	Interface d'extraction unique, deux implémentations
Données & gestion des erreurs	10	Nullabilité métier, transactions, schémas Zod
Expérience utilisateur	10	États vides, chargement, retour optimiste
Automatisation / IA pertinente	10	Règles déterministes + garde anti-hallucination
Documentation & installation	10	Quatre commandes, aucune dépendance externe
Priorisation & respect du temps	5	Fonctionnalités écartées explicitement documentées
Qualité de la démonstration	5	Traçabilité démontrable en direct

Lecture retenue : 60 points sur 100 récompensent un produit fini, lisible et robuste. L'IA n'en pèse que 10 et le sujet répète trois fois que le produit doit fonctionner sans elle. Elle est donc construite en dernier, et sacrifiée sans hésitation si le temps manque.

2. Étude comparative des stacks

Six options évaluées sur les critères pondérés par la grille d'évaluation. Notation de 1 à 5.

Stack	Vitesse	Lançable	Sécu. clé	Persist.	Archi.	UX	Install.	Live	Total
Next.js + Prisma + SQLite	4	5	4	4	4	4	5	2	32
Next.js + Supabase	4	2	4	5	4	4	2	5	30
FastAPI + SQLAlchemy + Jinja	3	4	4	4	4	2	4	2	27
Laravel / PHP	2	3	4	4	4	2	3	2	24
Vite + Express + PostgreSQL	2	2	4	4	3	3	2	2	22
FastAPI + front React séparé	1	2	4	4	3	3	1	1	19

Motifs d'écartement

Option écartée	Raison déterminante
Supabase	Le sujet exige une application lançable en local, et classe l'impossibilité de lancer en pénalité majeure. Supabase impose à l'évaluateur de créer un compte, un projet, de récupérer deux clés et de jouer un script SQL. Les clés ne peuvent pas être versionnées sans déclencher l'autre pénalité majeure. Le gain (lien live) ne compense pas le risque.
Vite + Express séparés	Deux <code>package.json</code> , configuration CORS, types dupliqués entre front et back, deux sections d'installation. Environ 45 minutes de plomberie non créditée, soit 12 % du budget total.
FastAPI + front séparé	Cumule le coût du découplage et celui du changement de langage. Le pire des deux mondes sur ce format.
Laravel / PHP	Écosystème solide mais mise en route plus lourde et rendu UI plus long à obtenir sans travail de design.
FastAPI + Jinja	Option de repli sérieuse et honnête. Serait retenue en cas de faible maîtrise de React : mieux vaut un code maîtrisé qu'une stack impressionnante mal tenue. Écartée ici pour l'UX, notée sur 10 points.

3. Stack retenue

Next.js 15 (App Router) · TypeScript · Prisma · SQLite · Tailwind CSS · shadcn/ui · Zod

Composant	Justification
Next.js 15 App Router	Un seul dépôt, un seul langage, un seul déploiement. Les Server Actions suppriment l'écriture d'une API REST et garantissent que le code d'accès aux données ne quitte jamais le serveur.
TypeScript	Les champs <code>owner</code> et <code>dueDate</code> peuvent être nuls par nature métier. Le compilateur force à traiter ce cas partout, ce qui est précisément l'exigence « ne pas inventer » du sujet.
Prisma	ORM schema-first à génération de code. Une source unique (<code>schema.prisma</code>) produit à la fois le SQL des migrations et les types TypeScript : les deux ne peuvent pas diverger. Fournit également les migrations versionnées et les écritures imbriquées transactionnelles.
SQLite	Base relationnelle complète contenue dans un fichier. Aucun serveur, aucun compte, aucune variable d'environnement. L'évaluateur lance l'application en quatre commandes, hors ligne. Neutralise la pénalité majeure la plus probable.
Tailwind + shadcn/ui	Composants copiés dans le dépôt, sans dépendance UI externe. Tableau, badges et sélecteurs propres en une vingtaine de minutes.
Zod	Validation à chaque entrée de Server Action. Une Server Action est un point d'entrée public : la validation n'est pas optionnelle.

Installation cible

```
npm install
npx prisma migrate dev
npm run seed      # charge le compte rendu Quizz+ du sujet
npm run dev
```

Le seed garantit que l'évaluateur ne voit jamais une application vide. Déploiement en ligne facultatif : Turso (SQLite hébergé, même dialecte) ou Railway avec volume, uniquement si le MVP est terminé.

4. Modèle de données

Deux modèles. Le choix structurant est de regrouper actions, points en attente et informations dans une table unique discriminée par le champ `kind`, plutôt que trois tables. Reclasser un élément devient ainsi une simple modification de champ.

```
model Meeting {
  id      String  @id @default(cuid())
  title   String
  meetingDate DateTime
  rawContent String
  reviewedAt DateTime? // null = en attente de validation
  createdAt DateTime @default(now())
  items   Item[]
}

model Item {
  id      String  @id @default(cuid())
  kind    String  @default("action")
  description String
  owner   String?
  dueDate DateTime?
  priority String?
  status  String?
  needsReview Boolean @default(false)
  reviewReason String?
  sourceExcerpt String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  meetingId String
  meeting Meeting @relation(fields: [meetingId], references: [id], onDelete: Cascade)

  @@index([kind, status])
  @@index([dueDate])
  @@index([meetingId])
}
```

Champ	Nullable	Rôle et justification
id	non	cuid() plutôt qu'un entier auto-incrémenté : n'expose pas le volume de données et n'est pas devinable dans l'URL.
kind	non	action / pending / info . Détermine la destination de l'élément dans l'interface.
description	non	Intitulé. Seul champ toujours renseigné, quel que soit le kind .
owner	oui	Nullabilité métier, non technique : le compte rendu de test contient des actions sans responsable nommé. Aucune valeur par défaut n'est jamais inventée.
dueDate	oui	Même logique. Une échéance absente reste absente et déclenche un signalement.
priority	oui	Uniquement pour les actions. Une information n'a pas de priorité.
status	oui	Uniquement pour les actions. Le retard n'est pas un statut : il est calculé.
needsReview	non	Booléen indexable, permet un filtrage direct « à confirmer ».
reviewReason	oui	Motif en clair destiné à l'utilisateur. Séparé du booléen pour conserver l'index et couvrir les signalements sans motif formulable.
sourceExcerpt	non	Phrase d'origine verbatim. Obligatoire au niveau du schéma : aucun élément ne peut exister sans preuve d'origine. Un ajout manuel porte une mention explicite.
updatedAt	non	Renseigné automatiquement par Prisma à chaque écriture.

Deux contraintes SQLite assumées

- **Pas de type enum**. `kind`, `priority` et `status` sont des `String`, contraints au niveau applicatif par des unions TypeScript (`as const`) alimentant à la fois les schémas Zod, les menus déroulants et les types. Source unique.
- **Pas de tri par priorité en base**. Un `orderBy` trierait alphabétiquement (`basse` < `haute` < `moyenne`), ce qui n'a aucun sens. Le tri par priorité se fait en TypeScript avec un rang explicite. Le tri par `dueDate` reste correct en base.

5. Architecture applicative

```
src/
  app/
    page.tsx           [serveur] liste des comptes rendus
    meetings/new/page.tsx [serveur] saisie
    meetings/[id]/page.tsx [serveur] validation OU fiche réunion
    dashboard/page.tsx   [serveur] vue consolidée filtrable
  components/          [client] formulaires, tableau, filtres
  lib/
    db.ts              singleton Prisma
    actions/           "use server" – mutations validées Zod
    extraction/
      types.ts          contrat commun
      rules.ts          extracteur déterministe
      ai.ts             extracteur LLM (V2)
      index.ts          sélection + repli
    constants.ts        KINDS, PRIORITIES, STATUSES
    validation/         schémas Zod partagés
```

Server Components et Server Actions

Mécanisme	Usage retenu
Server Component (défaut)	Toutes les pages. Appellent Prisma directement, sans <code>useEffect</code> ni route API ni état de chargement.
Client Component (<code>"use client"</code>)	Uniquement là où il y a de l'interactivité : formulaires, filtres, changement de statut.
Server Action (<code>"use server"</code>)	Toutes les écritures. Appelées comme des fonctions normales depuis le client, avec typage traversant la frontière réseau.
Route Handler	Aucune. Aucun consommateur externe au périmètre, donc pas de surface d'API à documenter et sécuriser inutilement.

Point de sécurité : une Server Action est un endpoint public, sollicitable avec n'importe quel payload. Être appelée depuis son propre formulaire ne la protège en rien. D'où la validation Zod systématique en entrée, sans exception.

Localisation de la clé IA

Variable d'environnement	Visible dans le navigateur
ANTHROPIC_API_KEY	Non — reste sur le serveur
NEXT_PUBLIC_ANTHROPIC_API_KEY	Oui — injectée dans le bundle JS

Le préfixe `NEXT_PUBLIC_` est le seul mécanisme d'exposition de Next.js. Le dépôt contient un `.env.example` vide ; `.env` et `prisma/dev.db` sont dans le `.gitignore`. Vérifiable en direct par affichage du code source de la page.

6. Extraction sans IA

Système à règles, environ 150 lignes de TypeScript, sans dépendance. Un compte rendu est un texte codifié : l'obligation s'y exprime presque toujours par les mêmes tournures.

Pipeline en quatre passes

1. **Segmenter** — découpage sur ponctuation forte, retours ligne et puces
2. **Classifier** — non-actions testées **en premier**, puis informations, puis déclencheurs d'action
3. **Extraire** — responsable, échéance, priorité, description nettoyée
4. **Signaler** — tout champ manquant produit `needsReview` et un motif

L'ordre des tests est le cœur de la logique métier. « Le budget n'a pas encore été validé » contient le mot « validé ». En cherchant les actions d'abord, on produirait l'action « valider le budget » avec un responsable inventé, exactement ce que le sujet interdit. En testant les non-actions d'abord, la phrase est écartée avant examen.

Dictionnaire	Motifs
Non-actions	n'a pas encore été validé/confirmé , ne pourra être ... qu'après , reste à valider , en attente de , sous réserve de
Informations	est prévue , aura lieu , a été rappelé
Déclencheurs d'action	doit / doivent / devra / devront , futur simple en -era / -eront , est chargé de , se charge de , prend en charge

Règles d'extraction des champs

Champ	Règle
Responsable	Mot capitalisé précédant un déclencheur, hors liste noire de mots vides. Collectifs (« l'équipe ») reconnus séparément et renvoyant <code>null</code> avec motif. Aucune valeur par défaut, jamais.
Échéance	Jour de la semaine résolu à sa prochaine occurrence strictement après la date de la réunion , jamais depuis la date du jour.
Priorité	Défaut <code>moyenne</code> . <code>haute</code> sur « urgent », « bloquant », ou échéance à moins de 48 h. <code>basse</code> sur « si possible », « à terme ». Champ assumé comme le plus subjectif, tranché par l'humain.
Description	Retrait du responsable, du modal et de la clause temporelle ; verbe remis à l'infinitif ; majuscule initiale.
Coordination	Une phrase liant deux obligations par « et » produit deux éléments. Le second hérite du responsable du premier.

Piège calendaire du sujet. Le compte rendu de test est daté du 27 juillet 2026, un lundi. « Avant jeudi » se résout donc au 30/07, « avant vendredi » au 31/07, « avant mercredi soir » au 29/07. Une résolution calculée depuis `new Date()` produirait des dates aberrantes, immédiatement repérables.

Résultat de référence sur le compte rendu Quizz+

Élément	Nature	Resp.	Échéance	Signalement
Rendre la version Android disponible pour les tests	action	—	—	Responsable collectif
Vérifier la configuration Play Store	action	Abdou	30/07	—
Corriger les erreurs signalées sur le classement	action	Awa	31/07	—
Préparer le message destiné aux testeurs	action	Mamadou	—	Échéance non précisée
Faire valider le message	action	Mamadou	29/07	Valideur non identifié
Date de lancement	pending	—	—	Conditionnée aux tests
Budget de la campagne	pending	—	—	Non validé
Réunion de suivi vendredi 15 h	info	—	—	—

Surcouche IA — évolution prévue, hors MVP

Même interface `Extractor` , implémentation différente. Prompt imposant du JSON strict et interdisant l'invention. Trois gardes côté serveur : parsing tolérant aux erreurs, validation Zod du JSON, puis **vérification que chaque `sourceExcerpt` cité existe réellement dans le texte source** — sinon l'élément est écarté. En l'absence de clé ou en cas d'échec, repli automatique et silencieux sur les règles.

7. Écran de validation

Principe fondateur : l'**extracteur propose, il n'enregistre jamais**. Rien n'est écrit en base avant action explicite de l'utilisateur.

L'extraction par règles étant déterministe, aucun état intermédiaire n'est stocké : tant que `reviewedAt` est nul, l'extraction est simplement rejouée à chaque affichage. Ni table de brouillon, ni cache.

Élément d'interface	Raison
Case à cocher par ligne	Décoché = non enregistré, mais la ligne reste visible. L'utilisateur voit ce qu'il écarte.
Tous les champs éditables	Description, responsable, échéance, priorité. L'extraction ne fait que pré-remplir un formulaire ordinaire.
Sélecteur de nature par ligne	Permet de reclasser un élément : la validation humaine porte sur la classification autant que sur les valeurs.
Compte rendu original accessible	Panneau dépliable permanent. Sans le texte complet, l'utilisateur ne peut pas juger de la fidélité d'une proposition.
<code>sourceExcerpt</code> sous chaque ligne	Traçabilité au niveau de la phrase. Vérification à deux niveaux avec le texte complet.
Badge de signalement avec motif en clair	Non pas une icône seule, mais la raison explicite du signalement.
Ajout manuel	Une action peut manquer à l'extraction sans être perdue.

L'enregistrement s'effectue via une Server Action unique, en transaction : la création des éléments et le marquage `reviewedAt` réussissent ou échouent ensemble. Aucune réunion ne peut être marquée validée sans ses éléments.

8. Tableau de bord

Vue	Contenu
<code>/meetings/[id]</code>	Tout ce qui provient de ce compte rendu — actions, points en attente, informations — plus le texte original.
<code>/dashboard</code>	Vue consolidée multi-comptes rendus, filtre « Compte rendu » réglé sur <i>Tous</i> par défaut.

Filtres : compte rendu, nature, statut, en retard, à confirmer, recherche texte. Les filtres transitent par l'URL (`?meeting=...&status=...`) plutôt que par un état React : liens partageables, bouton retour fonctionnel, et filtrage en base possible côté Server Component.

Modifications en ligne : statut et priorité, via Server Actions validées Zod, avec retour optimiste. Responsable et échéance modifiables via un panneau d'édition — un champ de date dans une cellule de tableau est inutilisable.

Retard : jamais stocké, toujours calculé (`dueDate < aujourd'hui` et `status ≠ terminé`). Un statut stocké se désynchronise dès que la date passe.

Tri : échéances les plus proches en tête, éléments sans date en fin de liste (`nulls: "last"`), puis priorité par rang explicite en TypeScript.

9. Plan de développement — 6 heures

Bloc	Durée	Contenu
1	45 min	Initialisation, schéma Prisma, migration, seed du compte rendu Quizz+
2	1 h 15	CRUD comptes rendus + extracteur par règles + tests
3	1 h 30	Écran de validation et enregistrement transactionnel
4	1 h 15	Tableau de bord : filtres, statuts, retards, compteurs
5	45 min	Surcouche IA — <i>uniquement si les blocs 1 à 4 sont terminés</i>
6	30 min	README et note de cadrage

Règle d'arbitrage : en cas de retard à la fin du bloc 4, le bloc 5 est abandonné. Perdre 10 points d'IA pour livrer un MVP fini est le bon calcul — et l'expliquer en démonstration rapporte les 5 points de priorisation. Le sujet le formule lui-même : une solution simple terminée vaut mieux qu'une solution ambitieuse inachevée.

10. Périmètre et limites assumées

Écarté volontairement

Authentification et multi-utilisateurs · notifications et rappels · export CSV/PDF · commentaires et fil de discussion · gestion de rôles · import de fichiers audio ou documents · historique des modifications.

Limites connues de l'extraction par règles

- Repose sur des tournures françaises standard ; un compte rendu télégraphique ou anglophone sera mal traité
- Ne résout pas les pronoms sur plusieurs phrases
- Le futur en `-era` peut produire des faux positifs (« sera », « verra »)
- Dates relatives limitées aux jours de la semaine et à quelques formules courantes
- Rendement décroissant : les vingt premières règles couvrent l'essentiel, les suivantes gèrent des cas rares et commencent à se contredire

Ces limites justifient l'évolution : la couche IA n'apporte pas un gain marginal, elle franchit un plafond que les règles ne peuvent structurellement pas franchir.

Suite proposée

Extraction sémantique par LLM avec vérification d'ancrage · authentification et espaces d'équipe · notifications d'échéance · intégration aux outils de réunion · historique et journal d'audit · migration PostgreSQL au-delà de quelques milliers d'éléments.

11. Contrôle final avant remise

Vérification	Risque couvert
Clone dans un dossier vierge, quatre commandes, application fonctionnelle	Application impossible à lancer
<code>.env</code> et <code>prisma/dev.db</code> absents du dépôt, <code>.env.example</code> présent	Clé API exposée
Application testée sans <code>ANTHROPIC_API_KEY</code>	Exigence « utilisable sans IA »
README : objectif, stack justifiée, installation, architecture, usage IA déclaré, limites	Absence de README
Note de cadrage : retenu, écarté, risques, suite	Priorisation (5 pts)
Temps effectif déclaré	Livable explicitement demandé
Historique Git lisible, commits progressifs	Historique minimal exigé
Chaque choix technique explicable en une phrase	Code non maîtrisé